Practical – 9

Practical Title: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition: Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

A. Theory

1. Is password_hash() used before storing passwords?
Yes. The PHP function password_hash() converts the user's password into a secure hash before storing it in the MySQL database. This prevents plain-text password storage and improves account security.

2. Is duplicate email/username checked?
Yes. Before inserting a new record, the PHP code checks whether the Student ID or email already exists in the students table. If a duplicate is found, registration is rejected and an appropriate error message is displayed.

3. Are frontend and backend validations consistent?
Yes. Both frontend HTML validation and backend PHP validation check the user's name, Student ID, email, phone number, program, semester and password. Password confirmation is also checked to ensure that both passwords match. Backend validation provides additional security even if frontend validation is bypassed.

B. Implementation/ Output/Result:

The student registration system was developed using HTML, PHP, MySQLi, and JavaScript. An HTML form collects student details such as name, Student ID, email, phone number, program, semester, and password. Frontend validation ensures that the entered information follows the required format, while JavaScript checks whether both password fields match.

On the backend, PHP performs server-side validation and checks the database for duplicate Student IDs and email addresses. The system connects to the existing studenthub_db database using the db_mysqli.php file. Before storing the password, PHP uses the password_hash() function to securely hash it. After successful validation and duplicate checking, the student's details are inserted into the existing students table using a MySQL INSERT query.

Finally, the system displays appropriate success or error messages. The implementation was tested by submitting registration details and verifying the newly inserted student record in phpMyAdmin.

C. Evidence of Work (Screenshots):

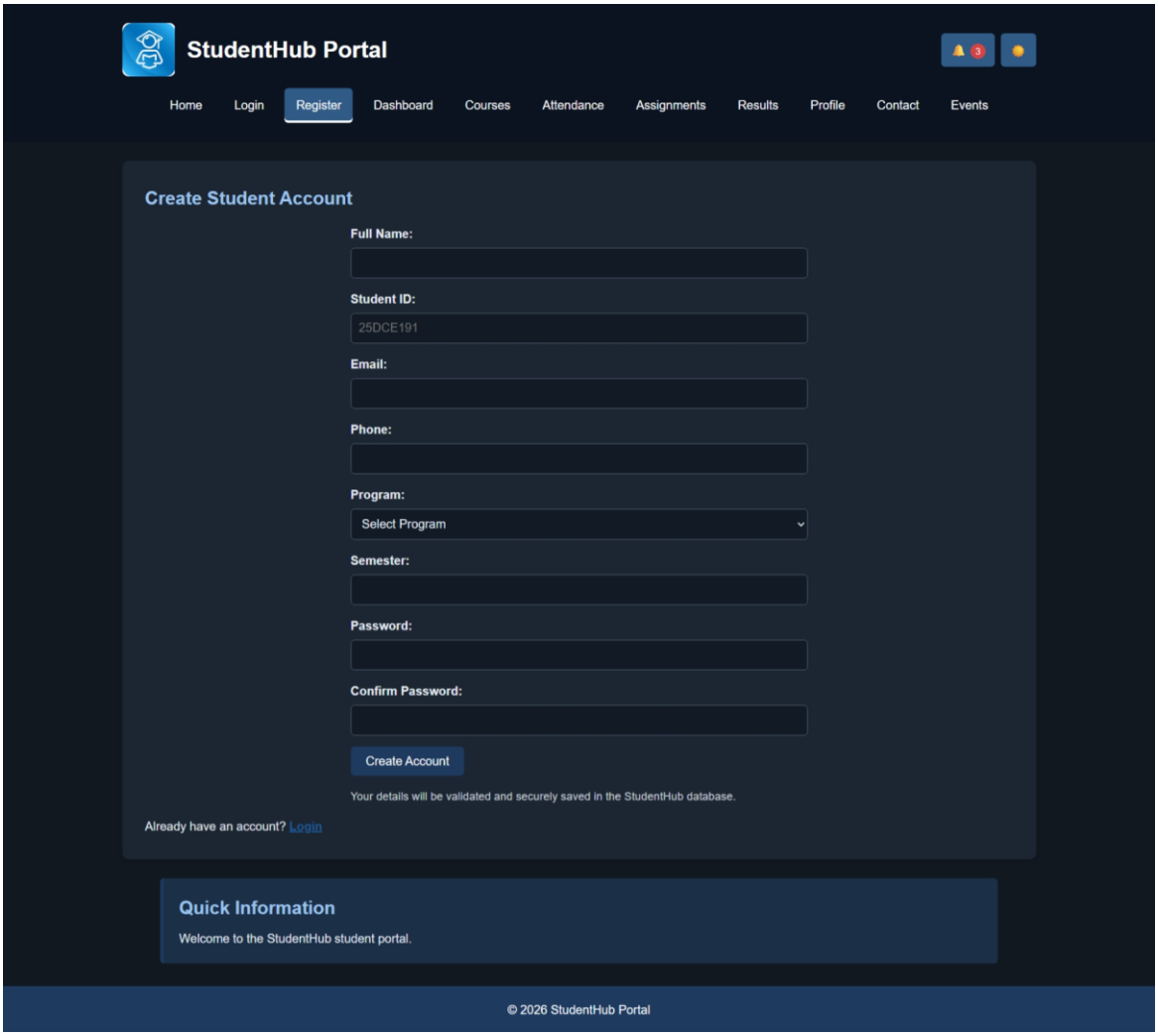


Fig1: Student Registration Form

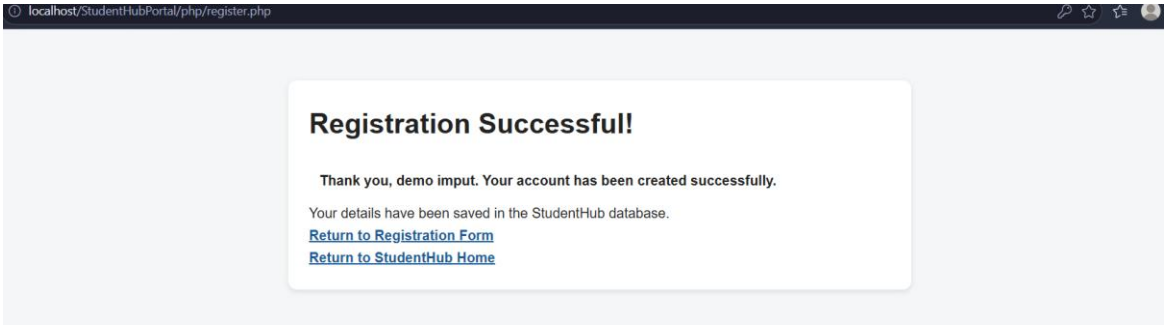


Fig2: Successful Registration

	student_id	name	email	phone	program	semester	password
Edit Copy Delete	25DCE001	Rahul Patel	rahul@gmail.com	9876543210	Computer Engineering	3	sample_hash_1
Edit Copy Delete	25DCE002	Priya Shah	priya@gmail.com	9876543211	Computer Engineering	3	sample_hash_2
Edit Copy Delete	25DCE003	Amit Desai	amit@gmail.com	9876543212	Information Technology	3	sample_hash_3
Edit Copy Delete	25DCE004	Neha Joshi	neha@gmail.com	9876543213	Computer Engineering	3	sample_hash_4
Edit Copy Delete	25DCE005	Karan Mehta	karan@gmail.com	9876543214	Information Technology	3	sample_hash_5
Edit Copy Delete	25DCE007	demo imput	demo@gmail.com	9874563211	B.Tech	4	$2y$10$Haca1b3gvnPizHBMtHA4TeD36NQTZen4mnfFI2EzhPJ...

Check all With selected: Edit Copy Delete Export

Fig3: Student Record in Database

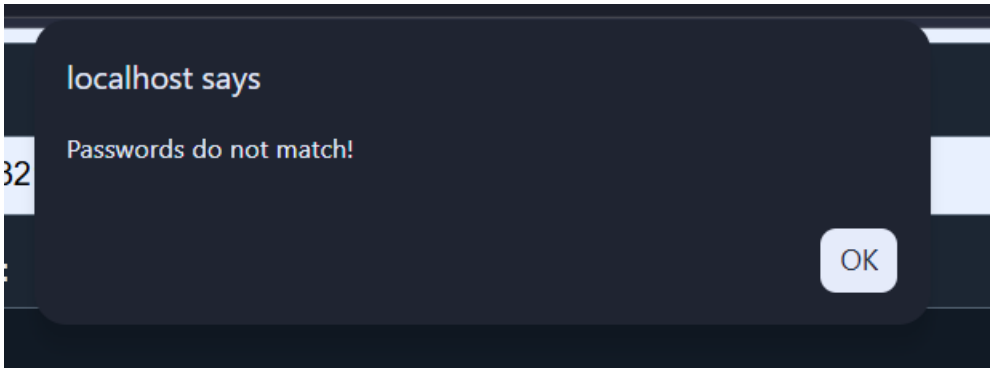


Fig4: Invalid Registration

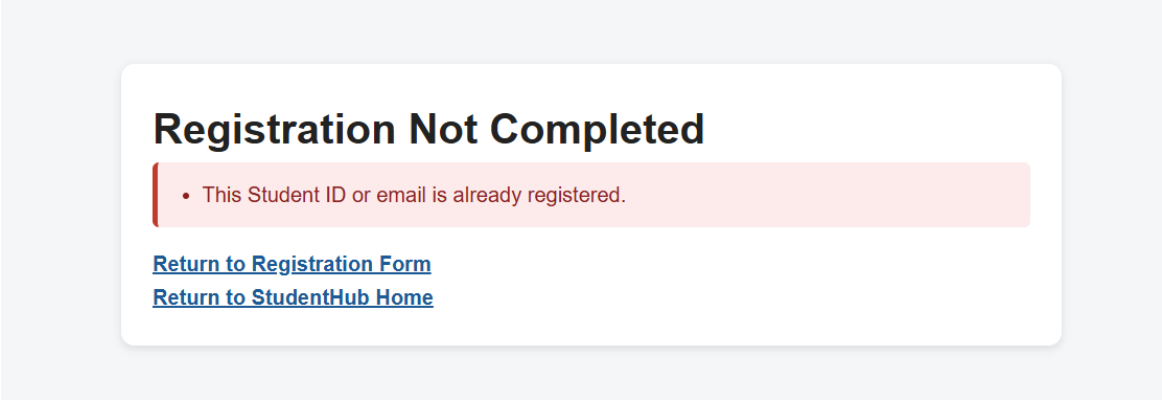


Fig5: Duplicate Registration

D. GitHub Repository Link:

aryan150806-wq/StudentHubPortal: Web Development Framework P

E. Challenges Faced:

1. Database Connectivity: Establishing a proper connection between PHP and the MySQL database using MySQLi.

2. Input Validation: Ensuring that all registration fields accept valid data on both frontend and backend.

3. Duplicate Record Detection: Checking existing Student IDs and email addresses to prevent duplicate registrations.

4. Password Security: Understanding and implementing password_hash() to store passwords securely instead of plain text.

5. Error Handling: Displaying appropriate success and error messages for valid, invalid, and duplicate registration attempts.

F. Learning Outcomes:

Through this practical, I learned to develop a student registration system using HTML, JavaScript, PHP, and MySQL. I understood how to perform frontend and backend validation, connect a PHP application to a MySQL database using MySQLi, and prevent duplicate Student IDs and email addresses. I also learned the importance of password security by implementing the password_hash() function and gained practical experience in inserting and verifying student records using phpMyAdmin.

Faculty Remarks: